

CSCI 5105 - Project #3: System Design and Report

Team Members: Quenton Ni, Cheston Opsasnick

Instructor: Jon Weissman

1. System Architecture Overview

We used a container-based distributed architecture built with Docker Compose rather than Kubernetes. The design cleanly separates request handling from replicated storage by placing a centralized Controller in front of a stateless Service tier and a Storage-node tier. Client requests enter through the Controller, which forwards work to Service Nodes, and those Service Nodes in turn coordinate with the replicated Storage Nodes. This gives the system horizontal scalability for request processing while keeping the data layer durable and fault tolerant through replication and primary-replica failover.

1.1 The Centralized Controller

The Controller acts as the central node in the system handling all requests. It receives client requests, selects a Service Node to handle each request, and continuously monitors node health so it can react to failures. It also manages storage membership and primary selection by tracking which storage node is currently primary and promoting a healthy replica when the primary becomes unavailable. In addition, it keeps metadata about active service nodes and storage nodes, and because this shared state can be accessed by background health-monitoring threads and request-handling paths, it uses locking to protect concurrent updates and keep the controller's view of the cluster consistent.

1.2 Service Tier vs. Storage Tier

- **Service Nodes (Stateless):** The Service Nodes are stateless request handlers. Their job is to validate and forward client operations, coordinate reads and writes against the appropriate storage target, and support persistent gRPC auction streams for live bid updates. Because they do not own state, they can be replaced or scaled independently without losing application data.
- **Data Replicas (Stateful):** The Storage Nodes hold the replicated auction data, maintain SQLite-backed persistence in docker volumes, and coordinate replication so that the primary and backup replicas stay consistent. Together, the service layer handles request

orchestration while the storage layer owns the replicated source of truth, which keeps the architecture modular, fault tolerant, and easier to scale.

2. Schema & Client API

```
// Example Item Schema representation
```

```
{  
  "item_id": "string",  
  "seller_id": "string",  
  "title": "string",  
  "category": "string",  
  "description": "string",  
  "starting_price": 0.0,  
  "current_price": 0.0,  
  "quantity": 1,  
  "status": "ACTIVE",  
  "version": 1,  
  "updated_at_epoch_ms": 1000  
}
```

```
// Example Bid schema
```

```
{  
  bid_id: "6cabdf0a-5fbc-454a-a2f2-83081bee61af",  
  item_id: "B000E31PNY-3-56c8c7",  
  bidder_id: "3a9abfee-9259-4936-b42f-b22a75909978",  
  amount: 1000,  
  item_version: 2,  
  created_at_epoch_ms: 1778099782901  
}
```

Auction Bidding:

The bidding protocol in the implementation is not bidirectional gRPC streaming in the strict sense. Instead, the client opens a long lived subscription through the JoinAuction RPC,

which is a server streaming RPC, and places bids through a separate PlaceBid call. The client side keeps the stream alive in a background listener thread, and the controller keeps subscriber queues in memory keyed by item_id. When a bid succeeds, the controller creates a BidEvent and broadcasts it to every active subscriber for that item, so listeners receive live auction updates as they happen. Clients can join an item's auction, continue listening for events, and receive each accepted bid in real time until they explicitly leave the stream.

3. Replication and Synchronization

3.1 Replication Strategy

We chose Primary-Backup replication because it is simpler to implement and fits the project's failover design. One storage node handles writes as the primary, and replicas stay synchronized for recovery. Reads are routed to the active storage target and return the current item state, while writes go to the primary first and are then replicated to backups so the replicas can take over if the primary fails.

3.2 Consistency Model

UpdateItemStorage follows an eventual consistency model. It commits the write on the primary first, then propagates that update to replicas. Until replication completes, replicas can temporarily return stale data, and if replication fails after the primary commit, the system can briefly diverge. Writes become consistent across nodes over time rather than being atomically visible everywhere at once upon every write.

4. Autoscaling Policy

Service-node layer scale-up is driven by the controller's autoscaling loop, which continuously tracks recent request throughput (requests per second over a rolling window), and converts that load into a desired service node count. Finally it adds nodes only when desired capacity exceeds current capacity. When scale-up is needed, the controller enforces a cooldown to prevent rapid churn, allocates the next sequential service-node ID, launches a new container on the project Docker network with the correct storage-target environment, waits for gRPC readiness, and finally appends the new target to the active routing pool so round-robin dispatch can start sending requests to it immediately.

- **Demand Metric:** Request Rate measured in the controller, averaged over a 15 second interval.
- **Scale-Up Behavior:** When the request rate crosses the preset threshold it spins up a new service node and adds it to the list to be used when new requests come in and gets selected in a round robin style.
- **Stability:** I have a background worker that checks every 5 seconds if the system should scale up or down. Also I average the requests over 15 seconds, and to prevent small spikes in traffic from scaling too quickly there is a set cooldown period.

5. Fault Tolerance and Recovery

5.1 Failure Detection

The controller runs a background health check every 2 seconds that calls `StorageNodeService.Heath()` with a 1 second RPC timeout and records if the health is ok or not. If not and if that target is the current primary it immediately runs a function promoting the first replica whose health shows ok on success. This means a single failed health check is enough to mark a storage node unhealthy and trigger promotion.

5.2 Catch-up and Recovery

When a replacement replica gets promoted to the primary, it should already be up-to-date with the old primary that failed. All storage requests will now be routed to this node, and it will continue to replicate itself to the last remaining backup storage-node.

6. Workload and System Evaluation

Used the given marketplace items as the primary data to be used to test the system.

Workload Scenario	Avg. Latency (ms)	Throughput (req/sec)	Active Service Nodes
Baseline Load	18.25	1	1
Burst Load	21.1	49.98	3

Workload Scenario	Avg. Latency (ms)	Throughput (req/sec)	Active Service Nodes
Auction Writes	20.57	10.3	1

6.1 Performance and Scalability Metrics

Autoscaling policy is working well, it scaled quickly, kept latency stable, and matched node count to observed throughput. After throughput goes back down, the extra service nodes get cleaned up as the system scales back down.

6.2 Fault Tolerance Testing

Logs from a fault test for Service node:

```
[test] controller health OK
[test] creating item before stopping a service node: svc-replace-c1205803
[test] item created successfully: svc-replace-c1205803
[test] initial active service node: service-node-3:50060
```

MANUAL STEP REQUIRED:

```
Please stop the node by running: docker stop service-node-3
Press Enter once you finish stopping the node...
[test] manual stop complete, waiting for controller to report a replacement node
[test] waiting for service node replacement after service-node-3:50060 (timeout=25.0s)
[test] last active service node observed: service-node-3:50060
[test] last active service node observed: service-node-4:50060
[test] replacement service node detected: service-node-4:50060
[test] replacement target confirmed: service-node-4:50060
[test] expected replacement host: greater than service-node-3
[test] replacement matches expected host service-node-4:50060
[test] verifying the item is still searchable after replacement:
service-node-replacement-dc81d0cd
[test] item still searchable after replacement: svc-replace-c1205803
```

Logs from a fault test for Storage node:

```
[test] create item before-fail-31b48934 result: True
```

[test] validating baseline item is replicated before stopping any node
[test] waiting for controller to report primary storage (timeout=5.0s)
[test] controller reports primary at node-level: storage-node-1:50051
[test] pre-stop replication check primary=storage-node-1:50051, replica=storage-node-2:50052
[test] comparing item before-fail-31b48934 between storage-node-1:50051 and
storage-node-2:50052
[test] item before-fail-31b48934 matches between nodes

MANUAL STEP REQUIRED:

Please stop the container with `docker stop storage-node-1`

Press Enter once you finish stopping the node...

[test] failover attempt 1: trying create after-fail-b848c7ad

[test] creating item via controller: after-fail-b848c7ad

[test] create item after-fail-b848c7ad result: True

[test] recovered: created item after-fail-b848c7ad

[test] waiting for controller to report primary storage (timeout=10.0s)

[test] controller reports primary at node-level: storage-node-2:50052

[test] promoted primary reported by controller: storage-node-2:50052

7. Design Tradeoffs and Lessons Learned

- **Consistency vs. Availability:**

We leaned toward availability during failures. The controller promotes a healthy replica when the primary becomes unhealthy, so the system tries to keep serving requests instead of blocking until all replicas are perfectly synchronized. That means consistency is slightly weaker than a strictly synchronous design. The primary applies writes first and then replicates them, so during a failure window a replica can lag behind. In practice, we preferred fast failover and continued operation over extra redundancy for consistency.

- **Simplicity vs. Performance:**

We made a few simplifying choices to keep the project overhead low. A single central controller handles routing, health checks, and scaling. Service-node placement is based on simple round-robin and hard-coded throughput thresholds. Replication uses direct RPC calls rather than a full consensus protocol. These choices made the system much easier to reason about and debug, but they also limited how much we could optimize for latency and correctness under failure.

- **Scalability vs. Overhead:**

Adding more service nodes helps only up to the point where routing and coordination

cost stay lower than how much they reduce the load. In our design, diminishing returns show up once the controller's overhead, health checking, and replication coordination start competing with the actual request load. Beyond that point, more nodes do not help much if the storage primary or replica syncing path is the bottleneck. With our current scaling rule, the benefits don't help much because the service nodes aren't the main bottleneck of the system, and the controller is already multithreaded.

8. Run Instructions

Running Instructions provided in the Repository's README.md